arXiv:2504.17827v5 [cs.NE] 29 Sep 2025

Evolution Meets Diffusion: Efficient Neural Architecture Generation

Bingye Zhou[1], **Caiyang Yu**[1], **Chenwei Tang**[1]

[1]Sichuan University
xxc8865@gmail.com, jerome0324@163.com, tangchenwei@scu.edu.cn

Abstract

Neural Architecture Search (NAS) has gained widespread attention for its transformative potential in deep learning model design. However, the vast and complex search space of NAS leads to significant computational and time costs. Neural Architecture Generation (NAG) addresses this by reframing NAS as a generation problem, enabling the precise generation of optimal architectures for specific tasks. Despite its promise, mainstream methods like diffusion models face limitations in global search capabilities and are still hindered by high computational and time demands. To overcome these challenges, we propose **E**volutionary **D**iffusion-based **N**eural **A**rchitecture **G**eneration (EDNAG), a novel approach that achieves efficient and network-free architecture generation. EDNAG leverages evolutionary algorithms to mimic the iterative refinement process of diffusion models, using fitness to guide the evolution of candidate architectures from diverse random populations to task-optimal solutions. Through this evolution-driven refinement process, our method achieves end-to-end acceleration of architecture generation, delivering optimal solutions in a fraction of the time required by conventional network-involved diffusion-based approaches. Extensive experiments demonstrate that EDNAG achieves SOTA performance in architecture optimization, with an improvement in accuracy of up to 10.45%. Furthermore, it eliminates the need for time-consuming training and boosts inference speed by an average of 50×, showcasing its exceptional efficiency.

Introduction

Neural Architecture Search (NAS) (Dong et al. 2024; Kang et al. 2023) represents an effective approach for the automated design of deep neural network architectures. However, conventional NAS methods are limited by the significant computational costs associated with exploring vast and intricate search spaces. Recently, the paradigm of Neural Architecture Generation (NAG) (Nasir et al. 2024; Lukasik, Jung, and Keuper 2022) has revolutionized this optimization challenge by reframing it from an optimization task into a generation task, thus mitigating the limitations imposed by large search spaces. The NAG framework leverages advanced generation techniques to directly produce optimized neural architectures that are finely tuned for specific downstream applications, offering more scalable and resource-efficient solutions.

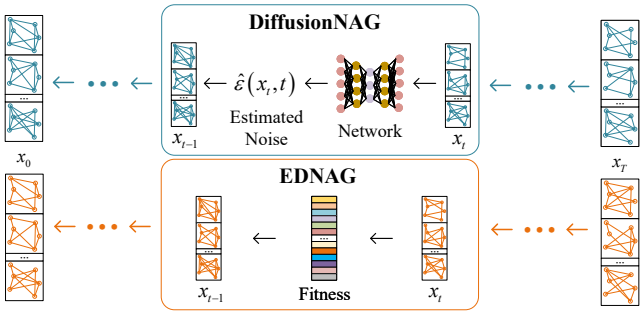


Figure 1: Overview of EDNAG. Unlike DiffusionNAG, which uses a trained noise prediction network to guide the denoising process, EDNAG employs a fitness-guided denoising strategy to derive the samples of next generation.

Previous architecture generation techniques can be categorized into two primary categories: Variational Auto-Encoders (VAE)-based (Lin et al. 2023) and Generative Adversarial Networks (GAN)-based (Huang et al. 2023) methods. VAE-based methods train an encoder to map architecture features into embedding representations and a decoder to generate them from the latent space. In contrast, GAN-based methods employ a generator to produce architectures that receive feedback from a discriminator. However, both VAE-based and GAN-based methods require the simultaneous training of two networks, leading to significant challenges in high computational complexity, training stability (Brock 2018), and generation quality. Recently, diffusion-based methods have demonstrated superior performance by training a single network and generating architectures with multiple denoising steps, thereby improving stability and generation quality. For example, DiffusionNAG (An et al. 2024) and DiNAS (Asthana et al. 2024) employ a graph diffusion framework, wherein the denoising process is iteratively conducted by a neural network trained to predict and remove noise. However, the need for thousands of such iterations results in significant computational overhead and slow generation speed. Therefore, diffusion-based methods still face two major challenges: **(1) Time-consuming network training**. Training UNet or Score-Net demands substantial computational resources, and retraining is always required when transitioning to different search

spaces; and **(2) Local optimality.** Naive diffusion models frequently converge to local optima, resulting in the generation of suboptimal architectures, especially when confronted with complex and extensive search spaces.

To the end, we propose a novel and efficient **E**volutionary **D**iffusion-based **N**eural **A**rchitecture **G**eneration Framework, called **EDNAG**. The core of EDNAG is to achieve a more globally optimal and efficient neural architecture generation.

we replace the traditional multi-step network-involved denoising process by introducing evolutionary algorithms to simulate the denoising process in diffusion models. This enables us to generate and refine architectures more efficiently without relying on repeated network inference, thereby avoiding time-consuming network training and significantly accelerating the generation process, as shown in Figure 1. Specifically, we propose a Fitness-guided Denoising (FD) strategy to produce a new generation of architectures from the previous ones in each denoising iteration, which prioritizes the exploration of high-potential regions. Through multiple denoising iterations incorporating selection strategies and random mutations, the architecture samples progressively evolve towards high-fitness regions while simultaneously avoiding local optima, thereby generating high-performing and globally optimalal neural architectures.

We perform extensive experiments within NAS-Bench-201 (Dong and Yang 2020) search space, demonstrating that EDNAG not only achieves SOTA performance in architecture generation with up to 10.45% improvement in accuracy, but also exhibits an average 50× speedup compared to previous baselines. Furthermore, we expand the search spaces to TransNASBench-101 (Duan et al. 2021), MobileNetV3 (Howard et al. 2019), DARTS (Liu, Simonyan, and Yang 2018) and AutoFormer (Chen et al. 2021a), demonstrating EDNAG's adaptability to extensive search spaces and its transferability to diverse downstream tasks.

In summary, our contributions are as follows.

- We propose a novel Evolutionary Diffusion-based framework for Neural Architecture Generation, called EDNAG, which simulates the denoising process in diffusion models within an evolutionary paradigm. EDNAG combines the superior generation quality of diffusion models with the global optimization capabilities of evolutionary algorithms.
- We introduce a Fitness-guided Denoising (FD) strategy, which utilizes fitness instead of trained networks to progressively generate offspring architecture samples from previous ones, achieving the first network-free neural architecture generation approach.
- Extensive experiments have demonstrated the SOTA performance of EDNAG in architecture generation with significantly fewer computational resources, as well as its outstanding adaptability and transferability.

Related Work

Neural Architecture Generation

Neural architecture generation (Dong et al. 2024; Kang et al. 2023) has evolved from VAE-based (Lin et al. 2023) to GAN-based (Huang et al. 2023) methods and has recently adopted diffusion-based (An et al. 2024) techniques. NAVIGATOR-D3 (Hemmi, Tanigaki, and Onishi 2024) is a VAE-based NAG method that extracts the graph features of architectures by an encoder and then reconstructs them through a decoder to generate optimal neural architectures. Following the paradigm of GAN, GA-NAS (Such et al. 2020) trains a discriminator to distinguish winning architectures and a generator to produce architectures based on the rewards provided by the discriminator, constituting an adversarial optimization problem. DiffusionNAG (An et al. 2024) represents the groundbreaking application of diffusion models in NAG, producing high-performing architectures through a denoising process with a trained score network. Rather than a continuous graph diffusion framework as in DiffusionNAG, DiNAS (Asthana et al. 2024) leverages a discrete graph diffusion model with a score network to generate neural architectures. However, both DiffusionNAG and DiNAS require numerous denoising iterations to generate architecture samples, along with frequent network inferences during iterations, leading to inefficiency in the generation process. Their iterative denoising process also suffers from slow convergence and a tendency to stagnate in local optima. Moreover, the aforementioned methods require the training process of one or more networks for generation tasks, which consume significant computational resources, while our method, EDNAG, uniquely achieves a network-free and training-free approach.

Evolutionary Algorithms

Evolutionary algorithms (Slowik and Kwasnicka 2020; Vikhar 2016), designed for optimization problems, serve as population-based heuristic paradigms simulating the natural evolution of species, which are appreciated for their global optimization capabilities. Recent studies have revealed potential connections between generative models and evolutionary algorithms, and have begun preliminary attempts to address challenging optimization problems with diffusion models. (Zhang et al. 2024) mathematically demonstrates the equivalence between diffusion and evolution, conceptualizing the diffusion process as reversed evolution and the denoising process as evolution. (Hartl et al. 2024) improves evolutionary algorithms by integrating diffusion models, enabling the retention of historical information across generations. Indeed, the aforementioned studies have largely focused on leveraging the properties of diffusion models to enhance evolutionary algorithms for optimization tasks. However, no prior work has attempted to integrate the principles of evolutionary algorithms into diffusion models for conditional generation tasks. Meanwhile, neural architecture generation, as a conditional generation task aimed at producing optimal neural architectures, naturally highlights the potential of integrating evolutionary algorithms into NAG, forming a foundational motivation for EDNAG.

Method

Preliminary

Denoising Diffusion Implicit Model (DDIM) (Song, Meng, and Ermon 2020) is a representative diffusion model. It involves a forward diffusion process, adding Gaussian noise to the original data, and a reverse denoising process, where the model learns to generate the original data by reversing the diffusion process.

In the forward process, we blend data with Gaussian noise, perturbing the original data x_0 to the final diffused data x_T as follows:

$$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1-\overline{\alpha}_t}\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I), \tag{1}$$

where $\overline{\alpha}_t = \prod_{i=1}^{T} \alpha_i$, representing the portion of data that remains undisturbed. In the reverse process, DDIM removes the noise from the randomly initialized data step by step, which is represented as:

$$x_{t-1} = \sqrt{\overline{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\overline{\alpha}_t}\hat{\epsilon}_\theta\left(x_t, t\right)}{\sqrt{\overline{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\overline{\alpha}_{t-1}-\sigma_t^2} \cdot \hat{\epsilon}_\theta\left(x_t, t\right)}_{\text{direction pointing to } x_t} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}. \tag{2}$$

where σ_t controls the degree of random perturbations $\epsilon_t \sim \mathcal{N}(0, I)$ during the denoising process. Notably, DDIM leverages a neural network ϵ_θ to predict perturbed noise $\hat{\epsilon}_\theta\left(x_t, t\right)$ at time step t, guiding the denoising process. By iteratively repeating this denoising process in Equation (2), DDIM progressively generates the final samples. Our method, EDNAG, performs architecture generation based on the denoising process of DDIM, but eliminates the need for a network to estimate $\hat{\epsilon}_\theta\left(x_t, t\right)$.

Evolutionary Diffusion-based NAG

Overview EDNAG generates neural architectures through an iterative denoising process. In each iteration, EDNAG first evaluates the fitness of the architecture population using a neural predictor, then employs the fitness-guided denoising (FD) strategy to generate offspring samples, and finally leverages optimization strategies to select architecture samples. The pipeline of EDNAG is described in Figure 2, and the pseudocode is displayed in Algorithm 1.

In the following sections, we first explain the fitness-guided denoising (FD) strategy. Next, we describe our optimization strategies for selecting architecture samples. More detailed derivations and explanations are provided in the Appendix.

Denoising Process of EDNAG FD strategy simulates the denoising process of DDIM to derive new samples x_{t-1} from previous samples x_t, generating offspring samples and driving the evolutionary process towards optimal solutions. By iteratively performing T denoising steps with FD strategy, the initial architecture samples x_T (random noise samples) can be progressively transformed into optimal architecture samples x_0.

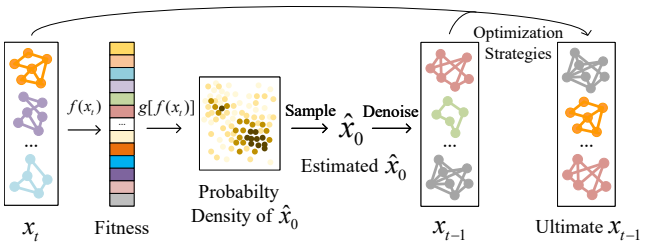


Figure 2: Fitness-guided denoising process. EDNAG generates neural architectures through iterative denoising process with FD strategy. In each iteration, FD strategy maps the fitness of x_t to the probability density of optimal samples x_0, subsequently uses the DDIM denoising formula to obtain x_{t-1}, and finally employs optimization strategies to select the final samples x_{t-1} from x_t and denoised x_{t-1}.

Algorithm 1: Neural Architecture Generation Process

Input: Population size N, number of nodes D_1, types of operations D_2, denoising steps T, fitness evaluator f, mapping function g.
Output: Generated optimal architectures x_0.

1: Sample $x_T \sim \mathcal{N}(0, I^{N \times D_1 \times D_2})$.
2: **for** t in $[T, T-1, \ldots, 1]$ **do**
3: Map fitness $f(x_t)$ to probability density $g[f(x_t)]$.
4: Denoise from x_t to $\hat{x}_{t-1}$ according to $g[f(x_t)]$.
5: Select final samples x_{t-1} at the $t-1$-th time step from parent x_t and denoised offspring $\hat{x}_{t-1}$ by optimization strategies.
6: **end for**
7: **return** x_0

For each denoising iteration, let $\hat{x}_0$ denotes the predicted x_0 in Equation (2):

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\overline{\alpha}_t}\hat{\epsilon}_\theta\left(x_t, t\right)}{\sqrt{\overline{\alpha}_t}}. \tag{3}$$

With Equation (3), the denoising process can be transformed into Equation (4).

$$x_{t-1} = \sqrt{\overline{\alpha}_{t-1}}\hat{x}_0 + \sigma_t \epsilon_t + \sqrt{1-\overline{\alpha}_{t-1}-\sigma_t^2} \cdot \frac{x_t - \sqrt{\overline{\alpha}_t}\hat{x}_0}{\sqrt{1-\overline{\alpha}_t}}. \tag{4}$$

Therefore, to perform the denoising process from x_t to x_{t-1}, it is only necessary to determine $\hat{x}_0$ (estimated optimal architectures).

According to the Bayes' theorem, the optimal architectures $\hat{x}_0$ can be estimated from the samples x_t at time step t and their fitness $f(x_t)$, as detailed below:

$$p(\hat{x}_0 = x | x_t) = \frac{p(\hat{x}_0 = x) \cdot p(x_t | x_0 = x)}{p(x_t)}, \tag{5}$$

where $p(x_t | x_0 = x)$ is derived from the forward diffusion process described by Equation (1), as shown below:

$$p(x_t | x_0 = x) \sim \mathcal{N}(x_t; \sqrt{\overline{\alpha}_t}x, 1-\overline{\alpha}_t). \tag{6}$$

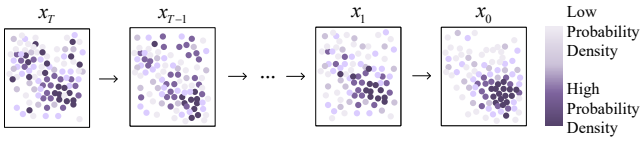


Figure 3: Denoising process of EDNAG. Individuals with higher fitness are more likely to be preserved throughout the denoising process, leading to an increased probability density in the final sample distribution.

In order to get $p(\hat{x}_0 = x)$, we should estimate $\hat{x}_0$ by mapping the distribution of x_t to the distribution of $\hat{x}_0$ with fitness guidance. Specifically, we view the denoising process of DDIM as the evolutionary process of GAs, both transforming samples from an initial random Gaussian distribution to an ultimate optimal distribution progressively (Zhang et al. 2024). We consider that individuals with higher fitness are more likely to be retained during evolution, resulting in a higher probability of appearing in the final samples $\hat{x}_0$, as shown in Figure 3. Therefore, a higher fitness corresponds to a higher probability density in $\hat{x}_0$. We model this relationship as a mapping function $g(x)$, which maps the fitness $f(x_t)$ of x_t to the probability density $p(\hat{x}_0)$ of $\hat{x}_0$:

$$p(\hat{x}_0 = x) = g[f(x)]. \tag{7}$$

We further define x_t as the previous samples consisting of N architectures, where $x_t = [\boldsymbol{x}_t^1, \boldsymbol{x}_t^2, \ldots, \boldsymbol{x}_t^N]$. Combining Equations (5) and (6), $\hat{x}_0$ can be estimated as follows:

$$\begin{aligned} \hat{x}_0 &= \frac{1}{p(x_t)} \sum_{\boldsymbol{x}_t^i \in x_t} g[f(\boldsymbol{x}_t^i)] \mathcal{N}(x_t; \sqrt{\overline{\alpha_t}}\boldsymbol{x}_t^i, 1-\overline{\alpha_t}) \boldsymbol{x}_t^i, \\ p(x_t) &= \sum_{\boldsymbol{x}_t^i \in x_t} g[f(\boldsymbol{x}_t^i)] \mathcal{N}(x_t; \sqrt{\overline{\alpha_t}}\boldsymbol{x}_t^i, 1-\overline{\alpha_t}), \end{aligned} \tag{8}$$

where $p(x_t)$ actually serves as a regularization term. In fact, $\hat{x}_0$ can be regarded as the weighted summation of each neural architecture $\boldsymbol{x}_t^i$ in samples x_t, where architectures with higher fitness have a greater influence on $\hat{x}_0$.

Finally, with Equations (4) and (8), we can denoise from x_t to x_{t-1} as follows:

$$\begin{cases} x_{t-1} = \sqrt{\overline{\alpha_{t-1}}}\hat{x}_0 + \sigma_t \epsilon_t \\ \qquad + \sqrt{1-\overline{\alpha_{t-1}} - \sigma_t^2} \cdot \dfrac{x_t - \sqrt{\overline{\alpha_t}}\hat{x}_0}{\sqrt{1-\overline{\alpha_t}}}, \\ \hat{x}_0 = \dfrac{1}{p(x_t)} \displaystyle\sum_{\boldsymbol{x}_t^i \in x_t} g[f(\boldsymbol{x}_t^i)] \mathcal{N}(x_t; \sqrt{\overline{\alpha_t}}\boldsymbol{x}_t^i, 1-\overline{\alpha_t}) \boldsymbol{x}_t^i, \\ p(x_t) = \displaystyle\sum_{\boldsymbol{x}_t^i \in x_t} g[f(\boldsymbol{x}_t^i)] \mathcal{N}(x_t; \sqrt{\overline{\alpha_t}}\boldsymbol{x}_t^i, 1-\overline{\alpha_t}). \end{cases} \tag{9}$$

By iteratively applying the FD denoising strategy across x_t, x_{t-1},..., x_0, which guides the evolution of architecture samples toward higher-fitness subspaces, EDNAG ultimately generates global-optimal neural architectures. In particular, ϵ_t represents stochastic noise, which serves as random mutations in GAs, thereby enabling the generation process to overcome local optima.

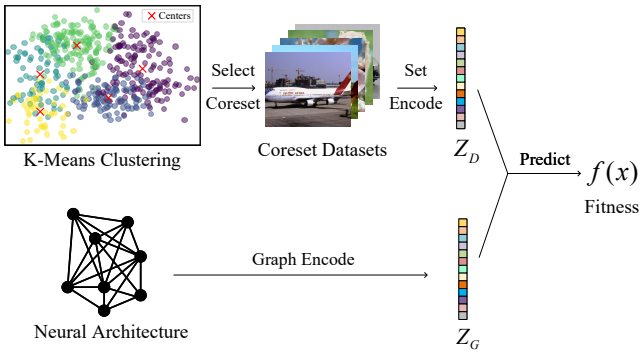


Figure 4: Neural predictor in EDNAG. During prediction, K-means clustering is applied to each class in $\mathcal{T}$, and the resulting centroids form the coreset $\mathcal{D}$. The set encoder $p(z_\mathcal{D}|\mathcal{D})$ and graph encoder $p(z_\mathcal{G}|\mathcal{G})$ embed $\mathcal{D}$ and neural architecture $\mathcal{G}$ into latent representations $z_\mathcal{D}$ and $z_\mathcal{G}$, respectively. The linear predictor $p(f(x)|z_\mathcal{D}, z_\mathcal{G})$ finally estimates the architecture's performance.

Optimization Strategies for Selection In each denoising iteration, we apply Equation (9) to denoise architecture samples from x_t to x_{t-1}. Subsequently, we employ optimization strategies (denoted as S) to select the final x'_{t-1} at time step $t-1$ from previous samples x_t and denoised samples x_{t-1}:

$$x'_{t-1} = S\left(x_t, x_{t-1}\right). \tag{10}$$

Specifically, we adopt an elitist strategy to retain the fittest architectures, a diversity strategy to preserve diverse architectures, and finally the roulette wheel strategy to select remaining architectures. During selection, 70% of individuals are chosen by roulette wheel selection, 10% by elitism, and the remaining 20% by the diversity strategy that selects the most distinct individuals based on the Euclidean distance. These optimization strategies, which simulate natural selection in GAs, ensure task-optimality while simultaneously improving the stability of the generation process and promoting diversity within the generated architecture samples.

Transferable Neural Predictor for Guidance

For task-specific conditional neural architecture generation, fitness evaluation is critical in guiding the denoising process of generation, which is conducted by a transferable neural predictor. Following DiffusionNAG, our neural predictor comprises three components: a set encoder $p(z_\mathcal{D}|\mathcal{D})$(Lee et al. 2019), a graph encoder $p(z_\mathcal{G}|\mathcal{G})$(Zhang et al. 2019), and a linear predictor $p(f(x)|z_\mathcal{D}, z_\mathcal{G})$.

In DiffusionNAG, the set encoder $p(z_\mathcal{D}|\mathcal{D})$ encodes a small subset of images randomly sampled from the task dataset for each prediction, which overlooks the uniformity of dataset distribution, resulting in biased and unstable predictions. To address these issues, we first select a coreset (Liu et al. 2023) from the task dataset and then feed it into the set encoder $p(z_\mathcal{D}|\mathcal{D})$. Specifically, during prediction, each class of images in the task dataset $\mathcal{T} = \left\{\left(x^i, y^i\right)\right\}^{|\mathcal{T}|}$ undergoes K-means clustering to select centroids of clusters as coreset $\mathcal{D} = \left\{\left(x^i, y^i\right)\right\}^{|\mathcal{D}|}$, where $|\mathcal{D}|$ represents the

number of samples per class and $|\mathcal{D}| \ll |\mathcal{T}|$. The entire process is detailed in Figure 4.

Notably, similar to previous methods, our neural predictor requires training. Although EDNAG shares this commonality with other NAG approaches based on generative models with proxy predictors, its key distinction and advantage lie in the generative model, which operates without score networks, thus significantly enhancing efficiency.

Experiments

We conduct sufficient experiments in the NAS-Bench-201, TransNASBench-101, MobileNetV3, DARTS, and AutoFormer search spaces. Subsequently, we perform ablation experiments on the FD strategy and further experiments on generation efficiency. For experiments in NAS-Bench-101 and NAS-Bench-301, baseline descriptions, hyperparameter settings, sensitivity analysis of parameters, comprehensive comparisons of GPU time across 14 baselines, and more in-depth experiments, please refer to the Appendix.

Experiments on Neural Architecture Generation

Experiment Setup We compare EDNAG with mainstream NAS methods, demonstrating its superior capabilities in architecture generation across various datasets. **One-shot NAS** performs NAS within sub-networks of a supernet, including weight inheritance-based and gradient-based methods. **BO-based NAS** employs Bayesian optimization techniques (e.g. Gaussian processes) to search for optimal neural architectures. **NAG** employs different generative models to directly generate neural architectures.

For the generation process, the denoising step of EDNAG is 100. The noise scale of diffusion ranges from $\alpha_T = 10^{-4}$ to $\alpha_0 = 1-10^{-4}$ and the noise scale of denoising is $\sigma = 0.8$. The population size (or batch size) is 100 for MobileNetV3 search space and 30 for other search spaces.

Experiments on NAS-Bench-201 Search Space NAS-BENCH-201 is a widely used cell-based search space comprising 15,625 distinct neural architectures. Within NAS-BENCH-201, we perform conditional generation of architectures across four task datasets and subsequently train the top-performing architecture to validate their performance, as shown in Table 1. Across diverse task datasets, EDNAG outperforms all previous methods, achieving SOTA performance and showcasing exceptional capability in generating optimal architectures. This is attributed to the FD strategy, which efficiently directs architecture sampling toward high-fitness subspaces, effectively steering the generation process toward global optima. Compared to DiffusionNAG, the previous SOTA baseline directly using naive diffusion models with limited optimization abilities, EDNAG excels with its superior global optimization capabilities while maintaining efficient generative abilities.

Moreover, EDNAG also significantly improves the efficiency of architecture validation by reducing the number of architectures (Archs) requiring training to achieve final accuracy. In all datasets, the final accuracies are obtained from the NAS-Bench-201 benchmarks (Dong and Yang 2020) by querying only the **top-1** generated architecture. The results

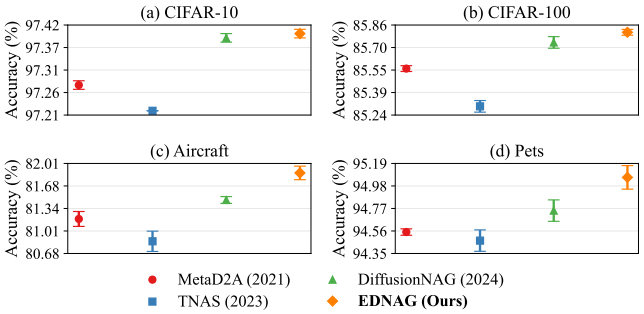


Figure 5: Experiments on the MobileNetV3 search space.

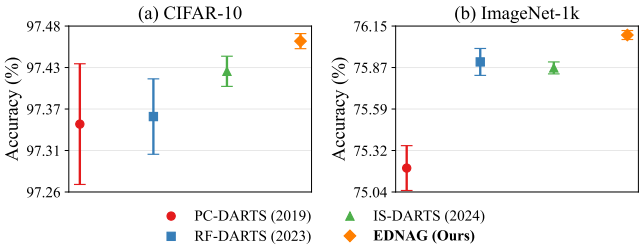


Figure 6: Experiments on the DARTS search space.

illustrate that the top-1 neural architecture identified by our neural predictor, attains optimal accuracy after real training, achieved by the application of coreset selection before dataset encoding. Unlike the random sampling method used in DiffusionNAG, our approach significantly enhances the reliability of the predicted accuracies.

Experiments on TransNASBench-101 Search Space TransNASBench-101 extends downstream tasks beyond basic image classification, encompassing a diverse set of 7 tasks: Object Classification (OC), Scene Classification (SC), Autoencoding (AE), Surface Normal Estimation (SE), Semantic Segmentation (SS), Room Layout Prediction (RP), and Jigsaw Puzzle Solving (JS). Moreover, it defines two types of search spaces: a cell-based micro search space and a stack-based macro skeleton search space, which encompass 4,096 and 3,256 unique architectures, respectively.

We conduct experiments in both micro-level and macro-level search spaces, as presented in Table 2. Across both search spaces, EDNAG demonstrates substantial performance enhancements of up to **7.02**% over prior baselines, consistently approaching global optimality across all downstream tasks while maintaining stability across three independent runs. These findings not only demonstrate the effectiveness of the FD strategy in improving global optimization capabilities for generative models, but also highlight the transferability of EDNAG for cross-task neural architecture generation within various downstream scenarios.

Experiments on MobileNetV3 and DARTS Search Spaces Due to the relatively limited scale of the above search spaces, our study further extends to more complex search spaces to demonstrate the scalability of EDNAG. MobileNetV3 is a large search space which encompasses

Type	Method	CIFAR-10		CIFAR-100		Aircraft		Pets	
		Acc. (%)	Archs	Acc. (%)	Archs	Acc. (%)	Archs	Acc. (%)	Archs
One-shot	RSPS (2019)	84.07±3.61	N/A	52.31±5.77	N/A	42.19±3.88	N/A	22.91±1.65	N/A
	SETN (2019)	87.64±0.00	N/A	59.09±0.24	N/A	44.84±3.96	N/A	25.17±1.68	N/A
	PC-DARTS (2020)	93.66±0.17	N/A	66.64±2.34	N/A	26.33±3.40	N/A	25.31±1.38	N/A
	DrNAS (2021b)	94.36±0.00	N/A	**73.51±0.00**	N/A	46.08±7.00	N/A	26.73±2.61	N/A
BO	BANANAS (2021)	**94.37±0.00**	46	**73.51±0.00**	88	41.72±0.00	40	40.15±1.59	17
	NASBOWL (2021)	94.34±0.00	100	**73.51±0.00**	87	53.73±0.83	40	41.29±1.10	17
	HEBO (2022)	94.34±0.00	100	72.62±0.20	100	49.32±6.10	40	40.55±1.15	18
	TNAS (2023)	**94.37±0.00**	29	**73.51±0.00**	59	**59.15±0.58**	26	40.00±0.00	6
NAG	DiNAS (2024)	**94.37±0.00**	192	**73.51±0.00**	192	-	-	-	-
	MetaD2A (2021)	**94.37±0.00**	100	73.34±0.04	100	57.71±0.20	40	39.04±0.20	40
	DiffusionNAG (2024)	**94.37±0.00**	1	**73.51±0.00**	2	58.83±3.75	3	41.80±3.82	2
	EDNAG (Ours)	**94.37±0.00**	**1**	**73.51±0.00**	**1**	**60.14±0.89**	**1**	**46.17±0.67**	**1**
	Global Best	94.37	N/A	73.51	N/A	-	N/A	-	N/A

Table 1: Experiments on NAS-BENCH-201 search space. Acc refers to the average accuracy over 3 runs with 95% confidence intervals, and Archs denotes the number of neural architectures actually trained to achieve this accuracy.

Level	Method	OC	SC	AE	SE	SS	RP	JS
		Acc. (%)	Acc. (%)	SSIM	SSIM	mIoU	L2 Loss	Acc. (%)
Micro	ZiCo (2024)	44.72±0.6	53.70±0.4	50.01±0.2	57.50±0.5	25.11±0.5	61.57±0.6	93.20±0.4
	GEA (2024)	45.98±0.2	54.85±0.1	57.11±0.3	58.33±1.0	25.95±0.2	59.93±0.5	94.96±0.2
	CATCH (2020)	45.27±0.5	54.38±0.2	56.13±0.7	56.99±0.6	25.38±0.4	60.70±0.7	94.25±0.3
	EDNAG (Ours)	**46.28±0.04**	**54.91±0.03**	**57.63±0.09**	**59.26±0.36**	**26.19±0.08**	**59.01±0.30**	**95.19±0.18**
	Global Best	46.32	54.94	57.72	59.62	26.27	58.71	95.37
Macro	REA (2019)	47.09±0.4	56.57±0.4	69.98±3.6	60.88±1.0	28.87±0.4	58.73±1.1	96.88±0.1
	PPO (2017)	46.84±0.4	56.48±0.3	70.92±3.2	60.82±0.8	28.31±0.5	58.84±1.1	96.76±0.2
	CATCH (2020)	47.29±0.3	56.49±0.3	70.36±3.0	60.85±0.7	28.71±0.4	59.37±0.6	96.82±0.1
	EDNAG (Ours)	**47.69±0.27**	**57.29±0.19**	**75.90±0.98**	**63.50±0.85**	**29.60±0.06**	**56.20±0.52**	**96.98±0.04**
	Global Best	47.96	57.48	76.88	64.35	29.66	55.68	97.02

Table 2: Experiments on TransNASBench-101 across seven tasks. Metrics are reported with 95% confidence intervals over 3 runs. For improved readability, the values in the table for SSIM, mIoU, and L2 Loss are scaled by 100.

over 10^{19} distinct neural architectures, and DARTS (Liu, Simonyan, and Yang 2018) is a well-known search space consisting over 10^{18} architectures. In addition, we extend our task dataset to ImageNet-1k (Deng et al. 2009), a classic large-scale dataset which spans 1,000 categories and contains over 10^6 images. In both search spaces, we report the average accuracy of different methods over 3 runs with with 95% confidence intervals.

We conduct experiments on four distinct task datasets within MobileNetV3, as shown in Figure 5, where EDNAG demonstrates an improvement in accuracy of up to **0.49**%. We also conduct experiments on CIFAR-10 and ImageNet-1k within DARTS, as presented in Figure 6, where EDNAG achieves an accuracy improvement of up to **0.24**%. This experiment not only demonstrates the superior ability of architecture generation for EDNAG but also confirms its adaptability and effectiveness in large-scale search spaces and task datasets.

Moreover, when transitioning to different search spaces, DiffusionNAG necessitates retraining the score network within the diffusion model to accommodate the new dimensionality of matrix, which is troublesome and time-consuming. In contrast, as a training-free NAG method, EDNAG adapts to new search spaces without training or retraining, incurring only negligible computational cost.

Experiments on AutoFormer Search Space With the increasing scale of model parameters, large-scale models are becoming increasingly common. However, existing NAS algorithms are rarely applied to large vision models. To address this gap, we conduct experiments for large-scale transformer-based vision models. AutoFormer search space is designed for vision transformer architectures and is classified into three levels: tiny, small, and base, facilitating the exploration of more than 1.7×10^{16} unique architectures. This design facilitates the exploration of architectures with parameter ranges from 4×10^6 to 75×10^6. The results in Table 3 demonstrate a significant improvement in

Type	Method	#Params (M)	Top-1 (%)	Top-5 (%)
Tiny	AutoFormer (2021a)	**5.7**	74.7	92.6
	TF-TAS-Ti (2022)	5.9	**75.3**	92.8
	PiMO-NAS-A (2024)	**5.7**	75.1	92.7
	EDNAG (Ours)	**5.7**	**75.3**	**92.9**
Small	AutoFormer (2021a)	22.9	81.7	95.7
	TF-TAS-Ti (2022)	22.8	81.9	**95.8**
	PiMO-NAS-A (2024)	21.4	81.6	95.5
	EDNAG (Ours)	**21.2**	**82.0**	**95.8**
Base	AutoFormer (2021a)	54.0	**82.4**	95.7
	TF-TAS-Ti (2022)	54.0	82.2	95.6
	PiMO-NAS-A (2024)	**51.7**	82.3	95.7
	EDNAG (Ours)	51.9	**82.4**	**95.8**

Table 3: Experiments on the AutoFormer search space. #Parameters, Top-1 Accuracy and Top-5 Accuracy are reported with 3 repeated runs with classification task on ImageNet-1k with 95% confidence.

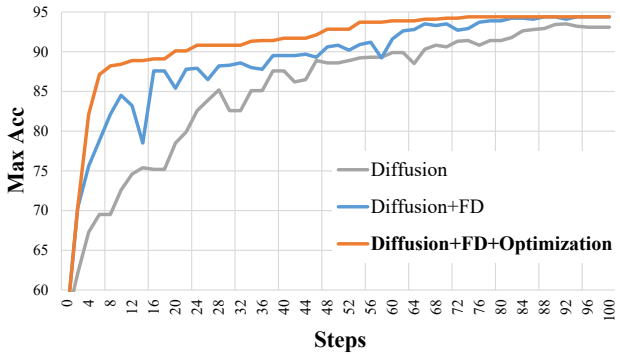


Figure 7: Comparison of architecture generation using Method 1, Method 2, and Method 3. we conducted 3 repeated experiments on CIFAR-10 within NAS-Bench-201.

accuracy for the generated architectures while achieving a smaller model size, underscoring EDNAG's strong performance and versatility in large-scale vision models. Notably, EDNAG achieves significant accuracy improvements across both high-parameter and low-parameter regimes, demonstrating its adaptability to varying computational constraints.

Ablation Study

To deeply analyze the contributions of the FD strategy and optimization strategies, we compare architecture generation under three distinct settings: (1) naive diffusion models, (2) diffusion models with the FD strategy, and (3) diffusion models incorporating both FD and optimization strategies, as illustrated in Figure 7. Method 1 frequently converges to local optima, yielding suboptimal architectures with an accuracy of 94.04 ± 0.16, while Method 2 notably improves accuracy to 94.34 ± 0.02 by introducing the FD strategy. Adding the optimization strategy in Method 3 further stabilizes generation and accelerates convergence, achieving

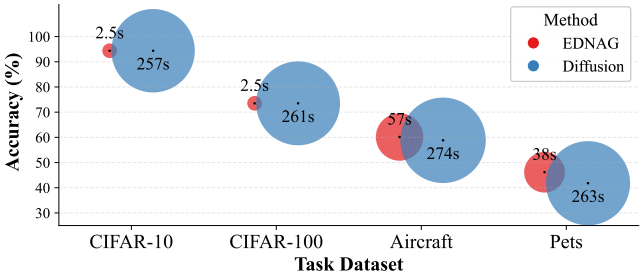


Figure 8: Comparison of GPU time costs. We report inference time for the architecture generation process over 3 runs. Circle radius represents generation time while vertical position indicates accuracies of generated architectures.

94.37 ± 0.00 accuracy. T-tests reveal statistically significant improvements: $p = 0.02334$ (Method 1 vs. Method 3) and $p = 0.019$ (Method 2 vs. Method 3). These results underscore the effectiveness of the FD strategy and the optimization strategy in EDNAG, demonstrating their pivotal roles in achieving task-optimal architecture generation.

Comparison of Generation Efficiency

As a pioneering network-free NAG approach, EDNAG achieves minimal computational resource consumption and rapid conditional generation. To demonstrate its efficiency, we compare EDNAG with DiffusionNAG in terms of computational costs during inference in NAS-Bench-201, as shown in Figure 8. Since EDNAG is free of training, it takes **0** GPU hours for the training process, while DiffusionNAG requires **3.43** GPU hours on a Tesla V100-SXM2. For the generation process, we evaluate the inference time (seconds) of both EDNAG and DiffusionNAG on an NVIDIA RTX 4090. EDNAG achieves speedups of up to **100**×, **100**×, **7**×, and **5**× compared to DiffusionNAG on CIFAR-10, CIFAR-100, Aircraft and Pets, respectively. Therefore, EDNAG enables neural architecture generation without time-consuming training and substantially reduces computational cost during inference. In contrast, DiffusionNAG relies on a complex score network at each denoising step, requiring frequent inference and resulting in significantly slower generation.

Conclusion

This study introduces EDNAG, an efficient and network-free neural architecture generation method. We propose a fitness-guided denoising strategy (FD) that maps fitness to a probabilistic space, guiding the DDIM-based denoising process for architecture generation. While EDNAG demonstrates strong performance, it remains dependent on a neural predictor for architecture evaluation, making the quality of generated architectures reliant on predictor accuracy. Future work may address this limitation by replacing the predictor-based evaluation step with evaluation-free strategies (e.g. zero-cost proxies) to guide architecture synthesis more autonomously. We believe that EDNAG will contribute to the development of NAS by reducing computational costs and accelerating practical application.

References

An, S.; Lee, H.; Jo, J.; Lee, S.; and Hwang, S. J. 2024. DiffusionNAG: Predictor-guided Neural Architecture Generation with Diffusion Models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*.

Asthana, R.; Conrad, J.; Dawoud, Y.; Ortmanns, M.; and Belagiannis, V. 2024. Multi-conditioned Graph Diffusion for Neural Architecture Search. *arXiv preprint arXiv:2403.06020*.

Brock, A. 2018. Large Scale GAN Training for High Fidelity Natural Image Synthesis. *arXiv preprint arXiv:1809.11096*.

Chen, M.; Peng, H.; Fu, J.; and Ling, H. 2021a. Autoformer: Searching transformers for visual recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*, 12270–12280.

Chen, X.; Duan, Y.; Chen, Z.; Xu, H.; Chen, Z.; Liang, X.; Zhang, T.; and Li, Z. 2020. Catch: Context-based meta reinforcement learning for transferrable architecture search. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIX 16*, 185–202. Springer.

Chen, X.; Wang, R.; Cheng, M.; Tang, X.; and Hsieh, C.-J. 2021b. DrNAS: Dirichlet Neural Architecture Search. In *International Conference on Learning Representations*.

Cowen-Rivers, A.; Lyu, W.; Tutunov, R.; Wang, Z.; Grosnit, A.; Griffiths, R.-R.; Maravel, A.; Hao, J.; Wang, J.; Peters, J.; and Bou Ammar, H. 2022. HEBO: Pushing The Limits of Sample-Efficient Hyperparameter Optimisation. *Journal of Artificial Intelligence Research*, 74.

Deng, J.; Dong, W.; Socher, R.; Li, L.-J.; Li, K.; and Fei-Fei, L. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, 248–255. Ieee.

Dong, X.; Kedziora, D. J.; Musial, K.; Gabrys, B.; et al. 2024. Automated deep learning: Neural architecture search is not the end. *Foundations and Trends® in Machine Learning*, 17(5): 767–920.

Dong, X.; and Yang, Y. 2019. One-shot neural architecture search via self-evaluated template network. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 3681–3690.

Dong, X.; and Yang, Y. 2020. Nas-bench-201: Extending the scope of reproducible neural architecture search. *arXiv preprint arXiv:2001.00326*.

Duan, Y.; Chen, X.; Xu, H.; Chen, Z.; Liang, X.; Zhang, T.; and Li, Z. 2021. Transnas-bench-101: Improving transferability and generalizability of cross-task neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 5251–5260.

Hartl, B.; Zhang, Y.; Hazan, H.; and Levin, M. 2024. Heuristically Adaptive Diffusion-Model Evolutionary Strategy. *arXiv preprint arXiv:2411.13420*.

Hemmi, K.; Tanigaki, Y.; and Onishi, M. 2024. NAVIGATOR-D3: Neural Architecture Search Using VarIational Graph Auto-encoder Toward Optimal aRchitecture Design for Diverse Datasets. In *International Conference on Artificial Neural Networks*, 292–307. Springer.

Howard, A.; Sandler, M.; Chu, G.; Chen, L.-C.; Chen, B.; Tan, M.; Wang, W.; Zhu, Y.; Pang, R.; Vasudevan, V.; et al. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, 1314–1324.

Huang, Z.; Wang, J.; Lu, X.; Mohd Zain, A.; and Yu, G. 2023. scGGAN: single-cell RNA-seq imputation by graph-based generative adversarial network. *Briefings in bioinformatics*, 24(2): bbad040.

Kang, J.-S.; Kang, J.; Kim, J.-J.; Jeon, K.-W.; Chung, H.-J.; and Park, B.-H. 2023. Neural architecture search survey: A computer vision perspective. *Sensors*, 23(3): 1713.

Lee, H.; Hyung, E.; and Hwang, S. J. 2021. Rapid neural architecture search by learning to generate graphs from datasets. *arXiv preprint arXiv:2107.00860*.

Lee, J.; Lee, Y.; Kim, J.; Kosiorek, A.; Choi, S.; and Teh, Y. W. 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, 3744–3753. PMLR.

Li, G. 2024. *Theoretically-grounded efficient deep learning system design*. Ph.D. thesis, The University of Texas at Austin.

Li, L.; and Talwalkar, A. 2019. Random search and reproducibility for neural architecture search. In *Uncertainty in Artificial Intelligence*, 367–377. PMLR.

Lin, Z.; Zhang, Y.; Duan, L.; Ou-Yang, L.; and Zhao, P. 2023. MoVAE: a variational AutoEncoder for molecular graph generation. In *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*, 514–522. SIAM.

Liu, H.; Simonyan, K.; and Yang, Y. 2018. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*.

Liu, Y.; Gu, J.; Wang, K.; Zhu, Z.; Jiang, W.; and You, Y. 2023. Dream: Efficient dataset distillation by representative matching. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 17314–17324.

Lopes, V.; Santos, M.; Degardin, B.; and Alexandre, L. A. 2024. Guided evolutionary neural architecture search with efficient performance estimation. *Neurocomputing*, 584: 127509.

Lukasik, J.; Jung, S.; and Keuper, M. 2022. Learning where to look–generative nas is surprisingly efficient. In *European Conference on Computer Vision*, 257–273. Springer.

Luo, G.; Li, H.; Chen, Z.; and Zhou, Y. 2024. Pareto-Informed Multi-objective Neural Architecture Search. In *International Conference on Parallel Problem Solving from Nature*, 369–385. Springer.

Nasir, M. U.; Earle, S.; Togelius, J.; James, S.; and Cleghorn, C. 2024. LLMatic: neural architecture search via large language models and quality diversity optimization. In *proceedings of the Genetic and Evolutionary Computation Conference*, 1110–1118.

Real, E.; Aggarwal, A.; Huang, Y.; and Le, Q. V. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence (AAAI)*.

Ru, B.; Wan, X.; Dong, X.; and Osborne, M. 2021. Interpretable Neural Architecture Search via Bayesian Optimisation with Weisfeiler-Lehman Kernels. In *International Conference on Learning Representations*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Shala, G.; Elsken, T.; Hutter, F.; and Grabocka, J. 2023. Transfer NAS with Meta-learned Bayesian Surrogates. In *The Eleventh International Conference on Learning Representations*.

Slowik, A.; and Kwasnicka, H. 2020. Evolutionary algorithms and their applications to engineering problems. *Neural Computing and Applications*, 32: 12363–12379.

Song, J.; Meng, C.; and Ermon, S. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*.

Such, F. P.; Rawal, A.; Lehman, J.; Stanley, K.; and Clune, J. 2020. Generative teaching networks: Accelerating neural architecture search by learning to generate synthetic training data. In *International Conference on Machine Learning*, 9206–9216. PMLR.

Vikhar, P. A. 2016. Evolutionary algorithms: A critical review and its future prospects. In *2016 International conference on global trends in signal processing, information computing and communication (ICGTSPICC)*, 261–265. IEEE.

White, C.; Neiswanger, W.; and Savani, Y. 2021. BANANAS: Bayesian Optimization with Neural Architectures for Neural Architecture Search. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Xu, Y.; Xie, L.; Zhang, X.; Chen, X.; Qi, G.-J.; Tian, Q.; and Xiong, H. 2020. Pc-darts: Partial channel connections for memory-efficient architecture search. In *International Conference on Learning Representations (ICLR)*.

Zhang, M.; Jiang, S.; Cui, Z.; Garnett, R.; and Chen, Y. 2019. D-vae: A variational autoencoder for directed acyclic graphs. *Advances in neural information processing systems*, 32.

Zhang, Y.; Hartl, B.; Hazan, H.; and Levin, M. 2024. Diffusion Models are Evolutionary Algorithms. In *International Conference on Learning Representations (ICLR)*.

Zhou, Q.; Sheng, K.; Zheng, X.; Li, K.; Sun, X.; Tian, Y.; Chen, J.; and Ji, R. 2022. Training-free transformer architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 10894–10903.